CSA1708-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Name: Krise Monali
Register No: 192519111

Program 1 : Python Program to Solve 8. Puzzle Problem

Aim

To develop a Python program that solves the 8-Puzzle Problem using a search algorithm (such as Breadth-First Search - BFS) to find the shortest sequence of moves from the initial state to the goal state.

Algorithm

1. Represent the puzzle as a 3×3 matrix.
2. Define the goal state.
3. Locate the empty tile (0).
4. Generate all possible moves (up, down, left, right).
5. Use Breadth-First Search (BFS) to explore possible states.
6. Keep track of visited states to avoid repeated exploration.
7. If the goal state is reached, display the solution path.
8. If all possibilities are exhausted, report that no solution exists.

S PUZZLE.py - C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py (3.14.4)

File Edit Format Run Options Window Help

```
from collections import deque

GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def get_neighbors(state):
    neighbors = []

    # Find the blank tile (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y],
            )

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors


def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == GOAL:
            return path + [state]

        if state in visited:
            continue
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py =======
Solution Found!

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
```

RESULT

Thus, the 8-Puzzle Problem solved successfully using Python.

Program 2: Python Program to Solve the 8-Queens Problem

Aim

To develop a Python program to solve the 8-Queens Problem using backtracking, placing eight queens on a chessboard so that no two queens attack each other.

Algorithm

1. Initialize an 8 × 8 chessboard.
2. Place one queen in each row.
3. Check whether the queen can be placed safely in a column.
4. Ensure that no two queens share the same column or diagonal.
5. If the position is safe, place the queen and move to the next row.
6. If no safe position is found, backtrack and try another column.
7. Repeat until all eight queens are placed.
8. Display the total number of solutions and the first solution.

mon12.py - C:/Users/Rahul/Documents/mon12.py (3.13.7)

File Edit Format Run Options Window Help

```
N = 8

def is_safe(board, row, col):
    """Check if a queen can be placed at (row, col)."""
    for r in range(row):
        c = board[r]

        # Same column or same diagonal
        if c == col or abs(c - col) == abs(r - row):
            return False

    return True


def solve(board, row, solutions):
    """Place queens row by row using backtracking."""
    if row == N:
        solutions.append(board[:])
        return

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(board, row + 1, solutions)
            board[row] = -1  # backtrack


def print_board(board):
    for row in range(N):
        print(" ".join("Q" if board[row] == col else "."
                       for col in range(N)))


if __name__ == "__main__":
    solutions = []

    solve([-1] * N, 0, solutions)

    print(f"Total solutions for {N}-Queens: {len(solutions)}\n")

    print("First solution:\n")
    print_board(solutions[0])
```

Ln: 42 Col: 2

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Rahul/Documents/mon12.py ==========
Total solutions for 8-Queens: 92

First solution:

Q . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
>>>
```

Ln: 17 Col: 0

Result

Thus, the 8-Queens Problem was solved successfully using Python and backtracking. The program found 92 solutions.

Program 3: Python Program to Solve the Water Jug Problem

Aim

To develop a Python program to solve the Water Jug Problem using Breadth-First Search (BFS) to obtain the target amount of water.

Algorithm

1. Define the capacities of the two jugs and the target amount.
2. Initialize both jugs as empty.
3. Use a queue to store the possible states of the jugs.
4. Generate the possible operations: fill a jug, empty a jug, or pour water from one jug into the other.
5. Check whether the target amount is reached.
6. Mark visited states to avoid repetition.
7. Continue exploring states using BFS until the target is reached.
8. Display the sequence of states leading to the solution.

mon12.py - C:/Users/Rahul/Documents/mon12.py (3.13.7)

File Edit Format Run Options Window Help

```
from collections import deque

def water_jug(cap_a, cap_b, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (a, b), path = queue.popleft()

        # Goal check
        if a == target or b == target:
            return path

        pour_a_to_b = min(a, cap_b - b)
        pour_b_to_a = min(b, cap_a - a)

        next_states = [
            (cap_a, b),                         # Fill jug A
            (a, cap_b),                         # Fill jug B
            (0, b),                             # Empty jug A
            (a, 0),                             # Empty jug B
            (a - pour_a_to_b, b + pour_a_to_b),  # Pour A -> B
            (a + pour_b_to_a, b - pour_b_to_a),  # Pour B -> A
        ]

        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

if __name__ == "__main__":
    cap_a, cap_b, target = 4, 3, 2

    solution = water_jug(cap_a, cap_b, target)

    if solution:
        print(f"Jugs: {cap_a}L and {cap_b}L, Target: {target}L\n")
        print("Steps (Jug A, Jug B):")
        for step, state in enumerate(solution):
            print(f"Step {step}: {state}")
    else:
        print("No solution possible.")
```

Ln: 45 Col: 3

```
>>> ========== RESTART: C:/Users/Rahul/Documents/mon12.py ==========
Jugs: 4L and 3L, Target: 2L

Steps (Jug A, Jug B):
Step 0: (0, 0)
Step 1: (0, 3)
Step 2: (3, 0)
Step 3: (3, 3)
Step 4: (4, 2)
>>>
```

Ln: 27 Col: 0

Result

Thus, the Water Jug Problem was solved successfully using Python and BFS. The target of 2 litres was obtained using a 4-litre jug and a 3-litre jug.

Program 4: Python Program to Solve the Crypt-Arithmetic Problem

Aim

To develop a Python program to solve the Crypt-Arithmetic Problem by assigning unique digits to letters so that the arithmetic equation is correct.

Algorithm

1. Define the words SEND, MORE, and MONEY.
2. Collect all unique letters from the words.
3. Generate possible digit assignments using permutations.
4. Ensure that each letter has a unique digit.
5. Prevent the first letter of each word from being assigned zero.
6. Convert the words into numbers using the digit assignments.
7. Check whether the equation SEND + MORE = MONEY is satisfied.
8. Display the digit assignments and the resulting numbers.

```
mon3.py - C:/Users/Rahul/Documents/mon3.py (3.13.7)
File Edit Format Run Options Window Help
from itertools import permutations

def solve_cryptarithmetic(word1, word2, result):
    # Collect all unique letters
    letters = sorted(set(word1 + word2 + result))

    if len(letters) > 10:
        return None

    # Letters that cannot be zero (first letter of each word)
    leading = {word1[0], word2[0], result[0]}

    def to_number(word, mapping):
        num = 0
        for ch in word:
            num = num * 10 + mapping[ch]
        return num

    for perm in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, perm))

        # Skip if a leading letter is assigned 0
        if any(mapping[ch] == 0 for ch in leading):
            continue

        if to_number(word1, mapping) + to_number(word2, mapping) == to_number(result, mapping):
            return mapping

    return None

if __name__ == "__main__":
    word1, word2, result = "SEND", "MORE", "MONEY"

    solution = solve_cryptarithmetic(word1, word2, result)

    if solution:
        print(f"Puzzle: {word1} + {word2} = {result}\n")
        print("Letter values:")
        for letter in sorted(solution):
            print(f"  {letter} = {solution[letter]}")

        n1 = int("".join(str(solution[c]) for c in word1))
        n2 = int("".join(str(solution[c]) for c in word2))
        n3 = int("".join(str(solution[c]) for c in result))
        print(f"\n  {n1}\n+ {n2}\n------\n {n3}")
    else:
        print("No solution found.")
```

```
========== RESTART: C:/Users/Rahul/Documents/mon3.py ==========
Puzzle: SEND + MORE = MONEY

Letter values:
  D = 7
  E = 5
  M = 1
  N = 6
  O = 0
  R = 8
  S = 9
  Y = 2

  9567
+ 1085
------
 10652
>>>
```

Result

Thus, the Crypt-Arithmetic Problem was solved successfully using Python.

Program 5: Python Program to Solve the Missionaries and Cannibals Problem

Aim

To develop a Python program to solve the Missionaries and Cannibals Problem using Breadth-First Search (BFS), ensuring that the missionaries are never outnumbered by cannibals on either riverbank when missionaries are present.

Algorithm

1. Define the total number of missionaries and cannibals as three each.
2. Initialize all people on the left bank and the boat on the left side.
3. Generate possible boat crossings carrying one or two people.
4. Calculate the new state after each crossing.
5. Check that the number of missionaries is never less than the number of cannibals on either bank when missionaries are present.
6. Use a queue to explore valid states using BFS.
7. Avoid revisiting states that have already been explored.
8. Continue until all missionaries and cannibals reach the right bank.
9. Display the sequence of crossings and the total number of crossings.

mon3.py - C:/Users/Rahul/Documents/mon3.py (3.13.7)

File Edit Format Run Options Window Help

```
        return False
    # Left bank
    if m > 0 and c > m:
        return False
    # Right bank
    m_right, c_right = total_m - m, total_c - c
    if m_right > 0 and c_right > m_right:
        return False
    return True

def solve(total_m=3, total_c=3, boat_cap=2):
    # State: (missionaries on left, cannibals on left, boat side: 0 = left, 1 = right)
    start = (total_m, total_c, 0)
    goal = (0, 0, 1)

    # All possible boat loads: (missionaries, cannibals)
    moves = [(m, c) for m in range(boat_cap + 1)
                    for c in range(boat_cap + 1)
                    if 1 <= m + c <= boat_cap]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) == goal:
            return path

        for dm, dc in moves:
            if boat == 0:   # boat moves left -> right
                new_state = (m - dm, c - dc, 1)
            else:           # boat moves right -> left
                new_state = (m + dm, c + dc, 0)

            if new_state not in visited and is_valid(new_state[0], new_state[1], total_m, total_c):
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))

    return None

if __name__ == "__main__":
    total_m, total_c = 3, 3
    solution = solve(total_m, total_c)

    if solution:
        print(f"Missionaries and Cannibals ({total_m}M, {total_c}C, boat holds 2)\n")
        print("State = (Missionaries left, Cannibals left, Boat side)\n")
        for step, (m, c, boat) in enumerate(solution):
            side = "Left" if boat == 0 else "Right"
            print(f"Step {step:2}: Left = {m}M {c}C | Right = {total_m - m}M {total_c - c}C | Boat on {side}")
        print(f"\nSolved in {len(solution) - 1} crossings.")
    else:
        print("No solution possible.")
```

Ln: 60 Col

```
========== RESTART: C:/Users/Rahul/Documents/mon3.py ==========
Missionaries and Cannibals (3M, 3C, boat holds 2)

State = (Missionaries left, Cannibals left, Boat side)

Step  0: Left = 3M 3C | Right = 0M 0C | Boat on Left
Step  1: Left = 3M 1C | Right = 0M 2C | Boat on Right
Step  2: Left = 3M 2C | Right = 0M 1C | Boat on Left
Step  3: Left = 3M 0C | Right = 0M 3C | Boat on Right
Step  4: Left = 3M 1C | Right = 0M 2C | Boat on Left
Step  5: Left = 1M 1C | Right = 2M 2C | Boat on Right
Step  6: Left = 2M 2C | Right = 1M 1C | Boat on Left
Step  7: Left = 0M 2C | Right = 3M 1C | Boat on Right
Step  8: Left = 0M 3C | Right = 3M 0C | Boat on Left
Step  9: Left = 0M 1C | Right = 3M 2C | Boat on Right
Step 10: Left = 0M 2C | Right = 3M 1C | Boat on Left
Step 11: Left = 0M 0C | Right = 3M 3C | Boat on Right

Solved in 11 crossings.
>>>
```

Result

Thus, the Missionaries and Cannibals Problem was solved successfully using Python and BFS. All three missionaries and three cannibals were transferred safely to the right bank in 11 crossings.